INSTITUTE OF SPACE TECHNOLOGY
KICSIT, Kahuta Campus

Dr. A. Q. Khan Institute of Computer Sciences and Information Technology

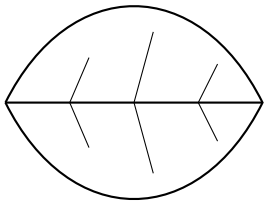
KICSIT Kahuta

MOBILE APPLICATION DEVELOPMENT

Assignment No. 2

LeafGuard AI

AI-Based Plant Leaf Disease Detection Application



Code Snippets and Explanation Report

Student Name:	Maira Masood
Roll Number:	242201061
Class:	BSCS-5B
Course:	Mobile Application Development
Instructor:	Sir Uzair
Submission Date:	05 November 2026

Contents

1 Introduction **1**

2 Objectives of the Project **1**

3 Application Workflow **1**

4 Dataset Information **2**

4.1 PlantVillage Dataset 2

4.2 Purpose of the Dataset 2

4.3 Image Input Requirements 2

4.4 Dataset Preparation 3

5 Google Colab and Model Training **3**

5.1 Why Google Colab Was Used 3

5.2 Model Training Process 4

5.3 Example: Loading and Preparing Images 4

5.4 Example: Model Training 5

5.5 TensorFlow Lite Conversion 6

5.6 Saving Class Labels 7

6 Android Application Development **7**

6.1 Android Studio 7

6.2 TensorFlow Lite in Android 7

6.3 TensorFlow Lite Dependency 8

6.4 Loading the TensorFlow Lite Model 8

6.5 Image Preprocessing 9

6.6 Model Output and Prediction 10

7 Application Screens and Their Explanation **11**

7.1 Screen 1: Splash Screen 11

7.2 Screen 2: Login Screen 12

7.3 Screen 3: Home Screen 13
7.4 Screen 4: Leaf Image Selection Screen 14
7.5 Screen 5: Image Preview and Detection Screen 15
7.6 Screen 6: Disease Result Screen 16

8 Complete Working of LeafGuard AI **16**

9 Testing and Verification **17**

10 Limitations **18**

11 Conclusion **18**

12 References **18**

1 Introduction

LeafGuard AI is an AI-based Android application designed to identify plant leaf diseases from an image. The application uses a trained machine learning model to analyze a leaf image and predict its possible disease category.

The main purpose of this project is to make plant disease identification easier through mobile technology. Instead of manually checking a leaf, the user can select an image from the device and view the model's predicted result.

The application uses:

- Android Studio for Android application development.
- Google Colab for model training and preparation.
- PlantVillage dataset for plant leaf image data.
- TensorFlow and TensorFlow Lite for machine learning.
- A trained model to predict one of the supported classes.

2 Objectives of the Project

The main objectives of LeafGuard AI are:

1. To develop a simple Android application for plant leaf disease detection.
2. To use artificial intelligence for image classification.
3. To train and prepare a model using Google Colab.
4. To convert the trained model into TensorFlow Lite format.
5. To allow the user to select one leaf image at a time.
6. To display the predicted disease category in the application.

3 Application Workflow

The application follows a simple step-by-step process:

1. The application starts with the splash screen.

2. The user enters an email and password on the login screen.
3. The user reaches the home screen.
4. The user selects one leaf image from the device.
5. The selected image is previewed and prepared for detection.
6. The application displays the predicted disease result.

Important: The application processes one leaf image at a time. The user can select another image after completing the previous detection.

4 Dataset Information

4.1 PlantVillage Dataset

The PlantVillage dataset is a collection of plant leaf images used for plant disease classification research and machine learning projects.

It contains images of healthy leaves and leaves affected by different diseases. These images help a model learn visual patterns that distinguish different plant conditions.

For this project, the model is configured to produce 38 output classes. These classes represent the categories supported by the model.

4.2 Purpose of the Dataset

The dataset is used to:

- Provide leaf images for model training.
- Help the model learn differences between leaf categories.
- Train the model to classify images into supported classes.
- Evaluate the model during the training process.

4.3 Image Input Requirements

The model input information inspected for this project is:

- Input shape: [1, 224, 224, 3]

- Input data type: float32
- Image height: 224 pixels
- Image width: 224 pixels
- Color channels: 3 (RGB)
- Output shape: [1, 38]
- Output data type: float32

The input shape means that the model receives one image at a time, resized to 224 by 224 pixels with three color channels.

The output contains 38 values. These values represent the model's scores for the 38 supported classes.

4.4 Dataset Preparation

Before training, the dataset images are prepared for the model. The general preparation process includes:

1. Loading images from the dataset.
2. Organizing images according to their class folders.
3. Resizing images to 224 by 224 pixels.
4. Converting images into numerical values.
5. Preparing training and validation data.

The exact dataset split and preprocessing settings should match the Google Colab notebook used for the final model.

5 Google Colab and Model Training

5.1 Why Google Colab Was Used

Google Colab is an online notebook environment that allows users to write and execute Python code in a browser.

It was used for the machine learning part of LeafGuard AI. It provides an environment for loading the dataset, preparing images, training the model, and converting the trained model into TensorFlow Lite format.

5.2 Model Training Process

The general model development process is:

1. Open Google Colab.
2. Load the PlantVillage dataset.
3. Prepare and resize the leaf images.
4. Create the image classification model.
5. Train the model using the prepared images.
6. Evaluate the model.
7. Convert the trained model into TensorFlow Lite format.
8. Export the model and its class labels for Android.

5.3 Example: Loading and Preparing Images

The following is an example of loading an image dataset from a folder in Google Colab. The folder path must be changed according to the actual dataset location.

Listing 1: Loading the image dataset

```
import tensorflow as tf

IMG_SIZE = (224, 224)
BATCH_SIZE = 32

dataset_path = "/content/PlantVillage"

train_data = tf.keras.utils.image_dataset_from_directory(
    dataset_path,
    validation_split=0.2,
    subset="training",
    seed=123,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE
)

validation_data = tf.keras.utils.image_dataset_from_directory(
    dataset_path,
    validation_split=0.2,
```

```
    subset="validation",
    seed=123,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE
)

class_names = train_data.class_names

print(class_names)
print("Number of classes:", len(class_names))
```

Explanation:

This code loads images from the dataset folder. It resizes the images to 224 by 224 pixels and creates training and validation datasets.

The class names are obtained from the folder names. The actual class order must be saved and reused by the Android application.

5.4 Example: Model Training

The following example demonstrates a MobileNetV2-based classification model. It should be used only if it matches the architecture and preprocessing of the model actually trained for this project.

Listing 2: Creating and training an example model

```
import tensorflow as tf
from tensorflow.keras import layers, models

number_of_classes = len(class_names)

base_model = tf.keras.applications.MobileNetV2(
    input_shape=(224, 224, 3),
    include_top=False,
    weights="imagenet"
)

base_model.trainable = False

model = models.Sequential([
    layers.Input(shape=(224, 224, 3)),
    layers.Rescaling(1./127.5, offset=-1),
    base_model,
```

```
    layers.GlobalAveragePooling2D(),
    layers.Dense(number_of_classes, activation="softmax")
])

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

history = model.fit(
    train_data,
    validation_data=validation_data,
    epochs=5
)
```

Explanation:

MobileNetV2 is a convolutional neural network architecture that can be used for image classification.

The model learns visual features from leaf images and produces a score for each supported class.

The number of output classes is obtained from the dataset folders. The number of epochs and other training settings must match the actual training notebook.

5.5 TensorFlow Lite Conversion

After training, a model can be converted into TensorFlow Lite format. TensorFlow Lite is designed to run machine learning models on mobile and embedded devices.

Listing 3: Converting a model into TensorFlow Lite

```
converter = tf.lite.TFLiteConverter.from_keras_model(model)

tflite_model = converter.convert()

with open("leaf_model.tflite", "wb") as file:
    file.write(tflite_model)

print("TensorFlow Lite model saved.")
```

Explanation:

This code converts the trained Keras model into a TensorFlow Lite model and saves it as `leaf_model.tflite`.

The converted file can then be added to the Android project's assets folder.

5.6 Saving Class Labels

The Android application needs the class labels in the same order used by the model during training.

Listing 4: Saving the class labels

```
with open("labels.txt", "w") as file:
    for label in class_names:
        file.write(label + "\n")

print("Labels saved successfully.")
```

Explanation:

This code saves the class names in a text file. The Android application can read this file to match the model's output position with the correct class name.

Important: The model and labels must come from the same training process. If their order does not match, the application may display an incorrect class name.

6 Android Application Development

6.1 Android Studio

Android Studio is the development environment used to create Android applications.

It provides tools for designing application screens, writing Java or Kotlin code, managing project files, and running the application on an Android device or emulator.

The LeafGuard AI application interface is developed in Android Studio.

6.2 TensorFlow Lite in Android

The TensorFlow Lite model is used for on-device image classification.

The model file is placed inside the Android project's assets folder. The application loads the model and uses an interpreter to perform inference.

An example project structure is:

```
app/
  src/
    main/
      assets/
        leaf_model.tflite
        labels.txt
      java/
      res/
      AndroidManifest.xml
```

6.3 TensorFlow Lite Dependency

The following is an example dependency for a Gradle project:

```
dependencies {
    implementation 'org.tensorflow:tensorflow-lite:2.16.1'
}
```

The dependency version should be checked against the version used in the actual Android project.

6.4 Loading the TensorFlow Lite Model

The following Java example shows the general idea of loading a model from the assets folder.

Listing 5: Loading the TFLite model

```
import android.content.Context;
import org.tensorflow.lite.Interpreter;
import java.io.FileInputStream;
import java.nio.MappedByteBuffer;
import java.nio.channels.FileChannel;

public class ModelHelper {

    public static MappedByteBuffer loadModel(
            Context context, String modelName)
            throws Exception {

        FileInputStream input = new FileInputStream(
            context.getAssets().openFd(modelName)
                .getFileDescriptor()
```

```
        );

        FileChannel channel = input.getChannel();

        return channel.map(
            FileChannel.MapMode.READ_ONLY,
            context.getAssets().openFd(modelName)
                .getStartOffset(),
            context.getAssets().openFd(modelName)
                .getDeclaredLength()
        );
    }

    public static Interpreter createInterpreter(
            Context context) throws Exception {

        MappedByteBuffer model =
            loadModel(context, "leaf_model.tflite");

        return new Interpreter(model);
    }
}
```

Explanation:

This example loads the TensorFlow Lite model from the assets folder and creates an interpreter.

The interpreter is responsible for running the model and producing its output. The code is an illustrative example and may need to be adjusted to match the actual Android project.

6.5 Image Preprocessing

Before an image is passed to the model, it must be prepared in the same way as the training images.

For the inspected model, the input shape is:

$$[1, 224, 224, 3]$$

This represents one RGB image with a height and width of 224 pixels.

If the model was trained using MobileNetV2 preprocessing with values scaled to the range [-1, 1], the corresponding normalization is:

Listing 6: Example image normalization

```
float normalizedValue =
    (pixelValue / 127.5f) - 1.0f;
```

Explanation:

The pixel value is converted into a normalized floating-point value.

This preprocessing example is correct only when it matches the preprocessing used during model training. The Android preprocessing must always match the training pipeline.

6.6 Model Output and Prediction

The inspected model output shape is:

$$[1, 38]$$

This means the model produces 38 output scores for one input image.

The application can identify the class with the highest output score and use the matching label.

Listing 7: Finding the highest-scoring class

```
int predictedIndex = 0;
float highestScore = output[0][0];

for (int i = 1; i < output[0].length; i++) {

    if (output[0][i] > highestScore) {
        highestScore = output[0][i];
        predictedIndex = i;
    }
}
```

Explanation:

The loop checks all output scores and finds the index with the highest value.

That index is used to find the predicted class name from the labels file.

The highest score is a model output score. It should not automatically be described as a verified probability or guaranteed diagnosis.

7 Application Screens and Their Explanation

This section explains the six main screens of LeafGuard AI. The screenshots are added using the image filenames `11.png` to `16.png`.

7.1 Screen 1: Splash Screen

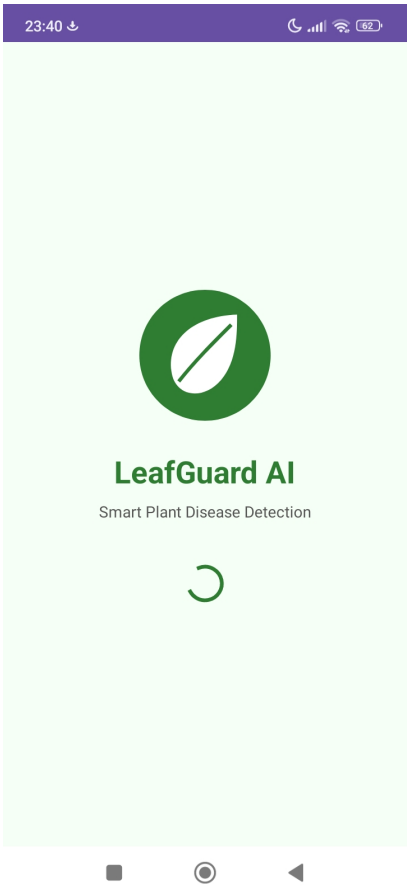


Figure 1: Splash Screen of LeafGuard AI

The splash screen is the first screen displayed when the application starts.

It introduces the application through its name and logo. It provides the first impression of the application before the user moves to the next screen.

Purpose:

- Introduces the application.
- Displays the application identity.
- Starts the application flow.

7.2 Screen 2: Login Screen

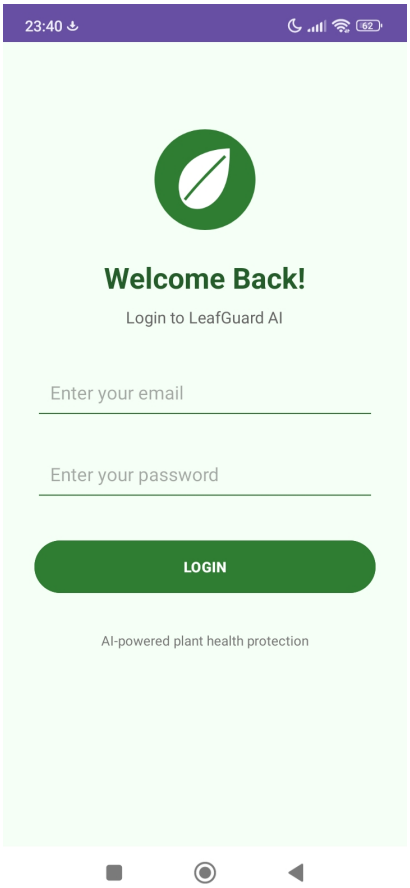


Figure 2: Login Screen of LeafGuard AI

The login screen allows the user to enter an email address and password.

It provides an interface for the user to proceed into the application.

Purpose:

- Accepts the user's email address.
- Accepts the user's password.
- Provides access to the next application screen.

7.3 Screen 3: Home Screen

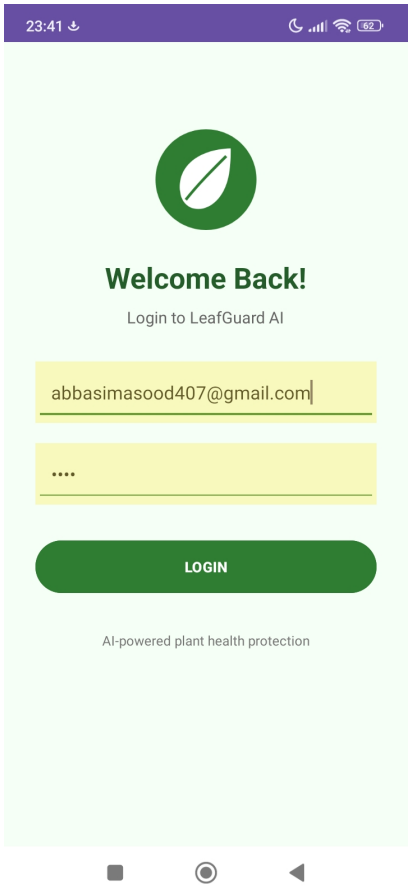


Figure 3: Home Screen of LeafGuard AI

The home screen is the main screen of the application after the login stage.

It provides the starting point for the leaf disease detection process. From this screen, the user can continue to the image selection stage.

Purpose:

- Acts as the main application screen.
- Provides access to the leaf detection process.
- Helps the user move through the application.

7.4 Screen 4: Leaf Image Selection Screen

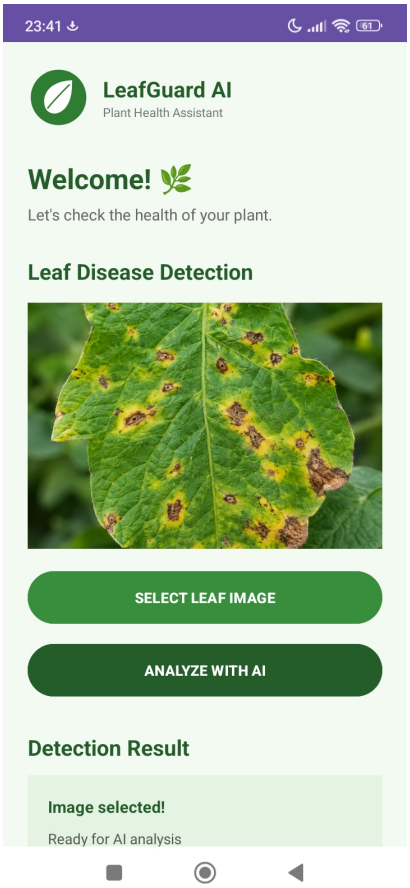


Figure 4: Leaf Image Selection Screen

This screen allows the user to select a leaf image from the device.

The application processes one image at a time. The user chooses a singl **Purpose:**

- Allows the user to choose a leaf image.
- Provides the image for the detection process.
- Starts the process using one selected image.

7.5 Screen 5: Image Preview and Detection Screen

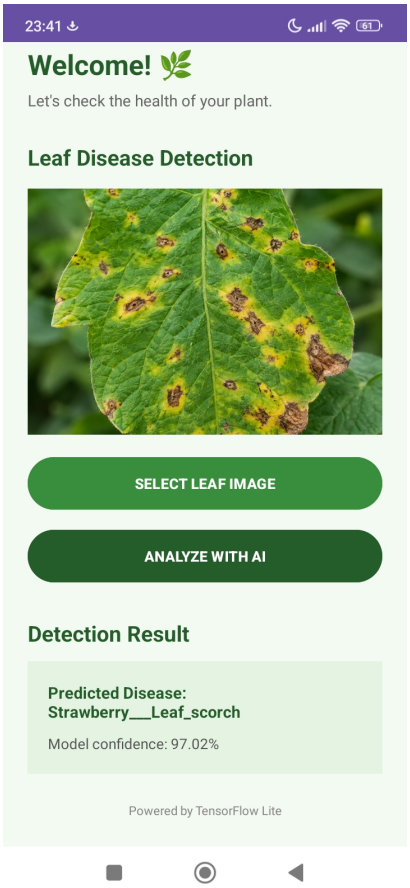


Figure 5: Selected Leaf Image Preview and Detection Screen

After selecting an image, the application displays or prepares the selected image for the detection stage.

This is the next step for the same image selected on Screen 4. It does not mean that two different images are processed together.

Purpose:

- Shows the selected leaf image.
- Allows the user to proceed with detection.
- Prepares the image for model processing.

7.6 Screen 6: Disease Result Screen

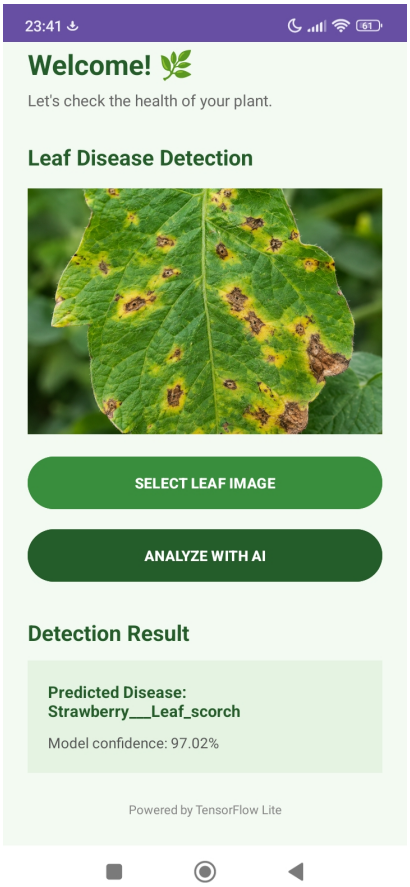


Figure 6: Disease Detection Result Screen

The result screen displays the predicted class for the selected leaf image.

The model produces output scores for its 38 supported classes. The application uses the predicted class index to identify the matching label.

Purpose:

- Displays the predicted disease or class name.
- Presents the result for the selected image.
- Completes one detection cycle.

After viewing the result, the user can select another image to perform a new detection.

8 Complete Working of LeafGuard AI

The complete application workflow is described below:

1. **Application Start:** The user opens LeafGuard AI and sees the splash screen.
2. **Login:** The user enters an email address and password.
3. **Home Screen:** The user reaches the main application screen.
4. **Image Selection:** The user selects one leaf image from the device.
5. **Image Preview:** The selected image is displayed or prepared for detection.
6. **Model Processing:** The image is resized and preprocessed according to the model's input requirements.
7. **Prediction:** The TensorFlow Lite interpreter processes the image and produces output scores.
8. **Result:** The application identifies the highest-scoring class and displays its label.

9 Testing and Verification

Testing is important to check whether the application screens and model work as expected.

The following points can be checked during application testing:

1. Check whether the application opens successfully.
2. Check whether the login screen is displayed.
3. Check whether the home screen opens.
4. Select one leaf image from the device.
5. Check whether the selected image is displayed correctly.
6. Check whether the model accepts the required image input.
7. Check whether the application displays the model's output label.

Testing Note:

Only report successful results for features that have actually been tested. If model inference is not yet connected to the Android interface, describe it as a planned or pending integration rather than a completed test.

10 Limitations

The application has some limitations:

- The model can classify only the classes included in its training dataset.
- The prediction depends on image quality and preprocessing.
- The model may produce incorrect predictions for unclear or unsupported images.
- The output is an AI prediction and should not be treated as a guaranteed plant diagnosis.
- The model's performance depends on the quality and variety of its training data.

11 Conclusion

LeafGuard AI is an AI-based Android application project developed to identify plant leaf disease categories using image classification.

The project combines Android application development with machine learning. Google Colab is used for dataset preparation, model training, and TensorFlow Lite conversion. Android Studio is used to develop the application interface and its mobile workflow.

The application allows the user to select one leaf image at a time and view the predicted result. The model input shape is [1, 224, 224, 3], and its output shape is [1, 38], representing 38 supported classes.

This project provides practical experience in Android development, dataset handling, machine learning, and mobile AI integration.

12 References

1. PlantVillage Dataset, plant leaf image dataset for plant disease classification.
2. TensorFlow Documentation: `https://www.tensorflow.org/`
3. TensorFlow Lite Documentation: `https://www.tensorflow.org/lite`
4. Google Colab: `https://colab.research.google.com/`
5. Android Developers: `https://developer.android.com/`
6. Project GitHub Repository: `https://github.com/maira123669/LeafGuard.AI`

7. Reference GitHub Repository: `https://github.com/Uzair390-Del/NumberGuessingAppTflite`

— End of Report —